Welcome to your next lesson: Objects.

I'm preparing your personalized lesson using what you've learned, what's next, and the best way to keep building your skills.

Step by step, you’re moving forward.

Ron, quick warm-up to get us moving 🔧

In JavaScript, we already know three basic value types:

number: 42, 3.14
string: "hello", 'Ron'
array: ["red", "green", "blue"]

Those are single values or ordered lists of values.

Now we’re adding objects, which are for grouping related values together under names — very similar to Python dicts, but with some JS twists. We’ll go step by step and compare to Python as we go.

To start simple:

Think of your profile: name, age, and city.

Would you naturally store that as:

a single number
a single string
an array
or a “dictionary-like” thing with labeled fields?

Just answer with the option that feels right, then we'll turn that idea into actual JavaScript code.

if it was going to have many such recorsd a dictionary for fast indexing, if its just for my own profile an array perhaps

Nice reasoning 👌 — using a dict for many records and maybe something lighter for a single one makes sense.

For a single profile in JavaScript, we'll almost always use that "dictionary-like" thing: an object.

In JS, a tiny profile object looks like this:

Javascript
```
const profile = {
  name: "Ron",
  age: 30,
  city: "London"
};
```
This is very close to a Python dict:

Python
```
profile = {
    "name": "Ron",
    "age": 30,
    "city": "London"
}
```
The big new idea:

An object groups related data as key: value pairs, like a Python dict.

Can you open the JS code editor and type only this code:

Javascript
```
const profile = {
  name: "Ron",
  age: 30,
  city: "London"
};
```

```
console.log(profile);
Run it (you don’t need to submit yet), and tell me what you see printed.

"name": "Ron",
"age": 30,
"city": "London"
}

Perfect—that output means your object literal is set up correctly 🎯


So right now we have this:

Javascript
const profile = {
  name: "Ron",
  age: 30,
  city: "London"
};
Think of it as:


Part    Meaning
profile variable holding the object
name    key (in JS we usually say property name)
"Ron"   value for that property

This matches Python’s dict idea closely.

Next step: reading one property.


In Python you’d do: profile["name"].

In JavaScript, one way is very similar:

Javascript
console.log(profile["name"]);
Can you add that console.log(profile["name"]); line under your existing code in the editor, run it, and tell me what the second line of output is?

Ron

```

Nice ✅ that means bracket notation with a string key feels natural already — just like Python.

Quick visual:

Python JavaScript Result
profile["name"] profile["name"] "Ron"

In JS, each key: value pair is usually called a property.

So we just read the name property of the profile object.

Next, I want to show you dot notation, which is a JS-only twist and a bit shorter than the bracket style.

Make sense so far:

You created an object and accessed profile["name"] to get "Ron"? 😊

If yes, I'll show you dot notation next.

Yes, that makes sense

Great 👍 let's add dot notation now.

In JavaScript, for simple keys like name, age, city, we can write:

Javascript
```
console.log(profile.name);
console.log(profile.age);
```
So:

profile["name"] ✅
profile.name ✅ (shorter JS style)

They do the same thing when the property name is a simple word (no spaces, doesn't start with a number).

Step by step:

In the editor, keep your existing code.
Add these two lines at the bottom:
Javascript
```
console.log(profile.name);
console.log(profile.age);
```
Run it (no need to submit yet).

You should now see four lines total. What are the last two lines of output?

Ron
30

Perfect ✅

So far you've seen:

profile["name"] → bracket notation (Python-style)
profile.name → dot notation (JS extra feature)

Both gave you "Ron".

Very short rule of thumb:

Use dot notation when the property name is a normal, fixed identifier (like name, age, city).
Use bracket notation when you have a string or a variable for the key.

We'll hit the dynamic/variable case later.

Next small step: let's change a property on the object.

In the editor, under your current code, add:

Javascript
```
profile.city = "New York";
console.log(profile.city);
```
Run it and tell me: what is the last line printed now?

New York

The . notation is much like python class where you define methods with the varaible names to enfoce not bypassing the encapsulatiion and try to access via cass.attrib

Exactly, that’s a sharp connection 🔎

In Python:

obj.attr usually means “this is a class instance attribute” (or method).
You could store data in a dict and do obj["attr"], but that feels different: more like a generic mapping, less like a structured object.

In JavaScript right now:

profile.name is just reading a property on a plain object.
There’s no class/encapsulation happening here yet — it’s more like a dict with a nicer syntax for simple keys.

So:

Idea Python JavaScript (today)
dictionary-like access profile["name"] profile["name"]
attribute-ish access obj.attr on class profile.name on object

Later, when you meet JS classes, obj.prop will be closer to your Python mental model. For now, it’s just the standard way to get/set object properties.

You already updated a property:

Javascript
profile.city = "New York";
console.log(profile.city); // New York
Next I want to show you adding a new property vs updating an existing one (same syntax, different effect).

Ready for a tiny add/update exercise with your profile object?

Yes, let's try it

Nice, let’s keep it small and clear 💪

Right now you have something like:

Javascript
```
const profile = {
  name: "Ron",
  age: 30,
  city: "London"
};

console.log(profile);
console.log(profile["name"]);
console.log(profile.name);
console.log(profile.age);

profile.city = "New York";
console.log(profile.city);
```
Two ideas:

Update existing property → same name, new value
```
profile.age = 31;
```
Add new property → brand new name
```
profile.country = "USA";
```

Both use dot notation.

Your turn:

In the editor, under your current code, add:
Javascript
```
profile.age = 31;        // update
profile.country = "USA"; // add
console.log(profile.age);
console.log(profile.country);
```
Run it (don’t submit yet).

What are the last two lines of output you see?

31
USA

Perfect 🎯

So you’ve now:

Created an object with { key: value }
Read properties with both profile["name"] and profile.name
Updated a property: profile.age = 31;
Added a property: profile.country = "USA";

That’s a big chunk of object basics already.

Next key idea I want to show you is dynamic access — using a variable for the property name, which you can’t do with dot notation.

Example shape (don’t type yet):

Javascript
const key = "name";
console.log(profile[key]); // uses the *value* of key
This is where bracket notation goes beyond Python-style dict access.

Make sense so far about:

dot notation for fixed names
bracket notation also able to take a variable?

If yes, I’ll have you try a tiny dynamic key experiment next.

Yes, let's try it

Awesome, let’s do dynamic access step by step 🧠

Right now you have profile with properties like name, age, city, country.

We’ll use a variable to choose which property to read.

In the editor, add this to the bottom of your code:

Javascript
const key = "name";
console.log(profile[key]);
Important:

key holds the string "name".
profile[key] means: “look up the property whose name is the value of key.”
Run it (no need to submit yet).

What is the very last line of output you see now?

Ron

Nice, that’s exactly what I wanted to see ✅

You just used dynamic property access:

Javascript
const key = "name";
console.log(profile[key]); // uses the value in key → "name" → profile["name"]
So we now have:

profile.name → fixed, known at write time
profile["name"] → same, but string
profile[key] where key === "name" → dynamic, chosen at runtime

Quick check in your own words (short is fine):

Why can we write profile[key] but not profile.key if we want to use the variable key?

because its a dynamic variable whos value isnt locked in until run time

Exactly 💡

profile.key would literally look for a property named "key".

But:

Javascript
const key = "name";
profile[key]; // looks for "name"
profile.key;   // looks for "key"
So:

Dot notation: property name is baked into the code.
Bracket notation: property name comes from a string/variable at runtime.

That “runtime choice” is the dynamic part.

Last important piece on objects for today: sometimes we also need to delete a property.

Example shape (don’t add yet):

Javascript
delete profile.country;
console.log(profile.country); // will show undefined
Ready to try a tiny delete step with your profile object?

Yes, let's try it

Great 😊 we’ll keep this one very small.

To delete a property in JavaScript, we use the delete operator:

Javascript
delete profile.country;
console.log(profile.country);
Step by step:

Go to the editor, at the very bottom of your code.
Add exactly this:
Javascript
delete profile.country;
console.log(profile.country);
Run it (no need to submit yet).

You should still see lots of earlier output, but I only care

about the last line now.

What is the last line printed?

undefined

Perfect 🎯

undefined here means: “this property doesn’t exist on the object anymore.”

You just:

Created an object with { key: value }
Read properties with dot and bracket notation
Updated and added properties
Used dynamic access with a variable key
Deleted a property with delete

That covers all the core object skills for this lesson ✅

Ron, before we wrap this one:

If I give you this object:

Javascript
```
const car = {
  brand: "Toyota",
  year: 2020
};
```
and this variable:

Javascript
```
const prop = "brand";
```
which one of these will read "Toyota"?

```
car.prop
car["prop"]
car[prop]
```

Just answer with 1, 2, or 3.

3

Exactly — 3 is the one that gives "Toyota" ✅

Because:

prop holds "brand"
car[prop] → car["brand"] → "Toyota"
car.prop / car["prop"] would look for a property literally named "prop".

You've nailed:

Object literal { key: value } ✅
Dot notation read/update ✅
Bracket notation & dynamic keys ✅

You're in great shape for this lesson.

Do you want to:

keep practicing objects a bit more (your own mini example), or move on to the next lesson in the track?
Move to the next lesson

Nice work today, Ron 🎉

You:

Built JavaScript objects with { key: value } and mapped them to your Python dict mental model.
Read and updated properties using dot notation (profile.name, profile.age).
Used bracket notation both with string keys (profile["name"]) and a dynamic variable key (profile[key]).
Added and deleted properties to see how objects change over time.

These object skills are going to show up everywhere in JavaScript from here on, so you've just unlocked a big piece

of the language 🔒

You’re all set to continue—go ahead and hit Next lesson when you’re ready, and I’ll keep walking with you step by step.

Nice catch, and that’s exactly the right detail to note 📝

You’re right:

Python dict:

```
profile = {
    "name": "Ron",
    "age": 30
}
```

Keys are strings, so they must be in quotes.

JavaScript object literal:

```
const profile = {
  name: "Ron",
  age: 30
};
```

Property names here look unquoted, but they’re really treated like string keys under the hood.

You can write them with quotes in JS too:

Javascript

```
const profile = {
  "name": "Ron",
  "age": 30
};
```

This works the same; it’s just less common when keys are simple identifiers.

For your notes, a good summary:

In object literals:

JS lets you omit quotes for simple identifier-like keys (name, age).
Python requires quotes around dict keys when they’re strings.

But when accessing:

Fixed key: profile.name or profile["name"]
Dynamic key: profile[key] (never profile.key)

Want a tiny table you can paste into your notes comparing Python vs JS for this?

pls

Love this level of detail, Ron — this is how you build a solid JS↔Python map 🧠✨

Here’s a compact table you can drop straight into your notes:

Markdown
Concept	Python (dict)	JavaScript (object)
Create mapping	`profile = {"name": "Ron", "age": 30}`	`const profile = { name: "Ron", age: 30 };`
Key / property syntax	Keys are strings → must use quotes	Simple keys can omit quotes in literals
Access fixed key	`profile["name"]`	`profile.name` or `profile["name"]`
Access dynamic key	`pro

```
const profile = {
  name: "Ron",
  age: 30,
  city: "London"
};

console.log(profile);
console.log(profile["name"]);
```

```
console.log(profile.name);
console.log(profile.age);
profile.city = "New York";
console.log(profile.city);

profile.age = 31;        // update
profile.country = "USA"; // add
console.log(profile.age);
console.log(profile.country);
const key = "name";
console.log(profile[key]);
delete profile.country;
console.log(profile.country);
console.log(profile[key])
```